Segundo Parcial · La ficha del personaje

Lenguajes de Programación II · 2026-2 · Martes 13 de octubre

Vale	**25 % de la asignatura**
Modalidad	Individual
Entrega	Todo **commiteado y subido a GitHub, en su rama**, cuando el docente cierre el examen

---

Lo que van a construir

La aplicación de Rick and Morty que armamos en clase ya muestra la **lista** de personajes, con paginación y estados. Cada tarjeta ya enlaza a `character.html?id=7`, pero **esa página está vacía**.

Su trabajo: construir la ficha del personaje.

La API: `https://rickandmortyapi.com/api/character/7` (el `7` es el número del personaje). Esta dirección devuelve **un** personaje, no una lista:

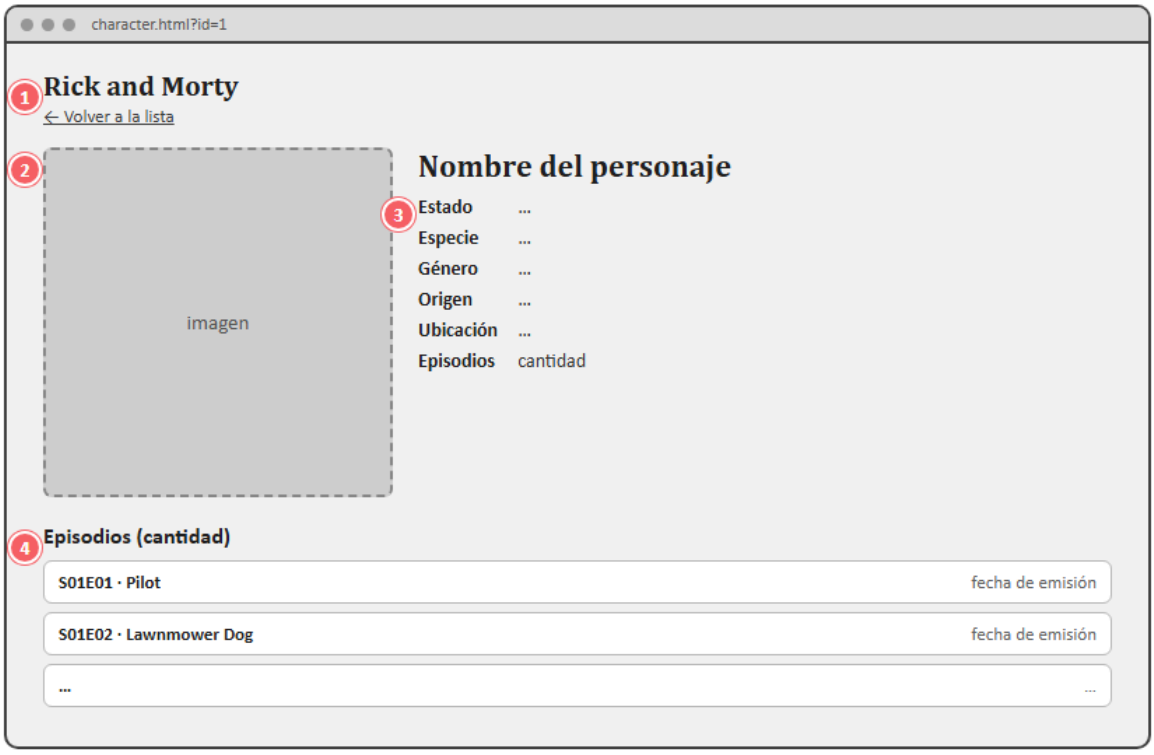
```
{ "id": 7, "name": "Abradolf Lincler", "status": "unknown", "species": "Human",
  "gender": "Male", "origin": { "name": "Earth (Replacement Dimension)", "url": "…" },
  "location": { "name": "Testicle Monster Dimension", "url": "…" },
  "image": "https://rickandmortyapi.com/api/character/avatar/7.jpeg",
  "episode": [ "https://rickandmortyapi.com/api/episode/10", "…" ] }
```

Una sugerencia de cómo podría quedar

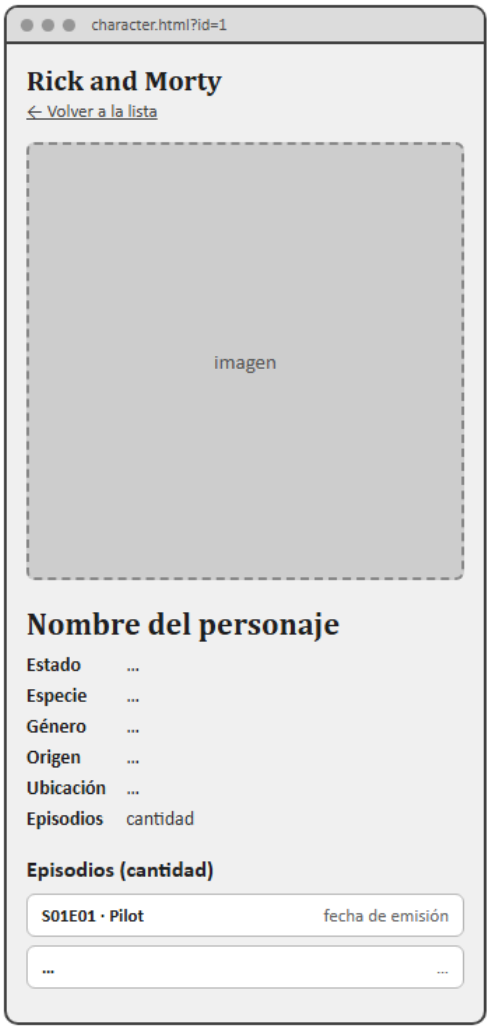
El siguiente *mockup* es **una sugerencia de cómo debería quedar la ficha**. Es solo una guía de la disposición: **ustedes tienen total libertad para hacerla de acuerdo con su creatividad.** Los colores, las fuentes, los espacios, la forma de las imágenes y los textos son suyos.

Lo que sí tiene que cumplirse es lo que pide el enunciado: los datos, los episodios, los cuatro estados y que la ficha se adapte al celular.

EN UN COMPUTADOR



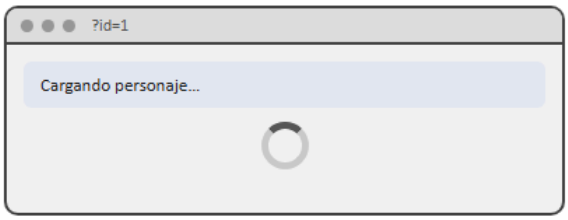
EN UN CELULAR (375 DE ANCHO)



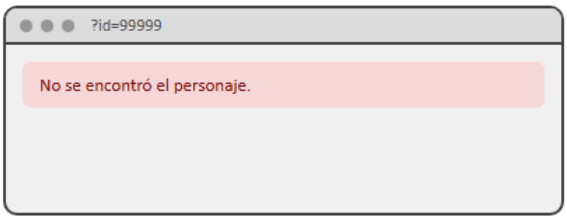
1 Encabezado y enlace «← Volver a la lista» 2 Imagen 3 Los datos del personaje (estado, especie, género, origen, ubicación y cuántos episodios) 4 Lista de episodios: código y nombre de cada uno

En el celular, la imagen y los datos quedan uno debajo del otro, sin desplazamiento horizontal. El título de la pestaña del navegador es el nombre del personaje.

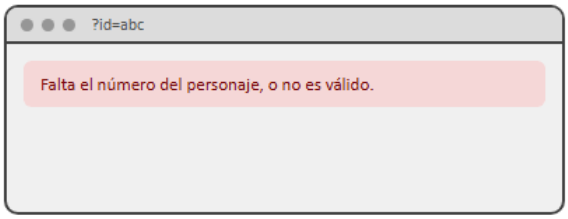
LO QUE SE VE EN CADA CASO



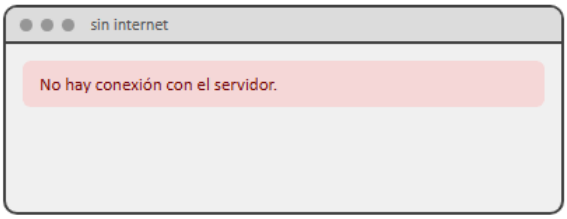
Cargando. Mensaje o spinner. Desaparece siempre al terminar.



No existe. El número es válido, pero no hay personaje.



Número ausente o inválido. Sin pedir nada a la API.



Sin conexión. Un mensaje distinto de los anteriores.

Los textos de los mensajes son un ejemplo: cada uno escribe los suyos, en español, siempre que los cuatro casos se distingan entre sí.

Cómo empezar (todos parten del mismo código)

0. **Compruebe que Git sabe quién es usted.** Cada commit lleva su nombre como autor, y así se identifica su trabajo:

```
git config --global user.name
git config --global user.email
```

Deben salir **su nombre completo real** y el correo de su cuenta de GitHub. Si salen vacíos o con otro nombre, configúrelos (la *Chuleta de Git* explica cómo).

1. **Clone el repositorio en una carpeta nueva**, aunque ya lo tenga: así nadie parte de código diferente.

```
git clone https://github.com/ClasesUniremington/LenguajesDeProgramacion2_2026-2.git
```

2. Entre a la carpeta y **cree una rama con su nombre completo**, en minúsculas, sin tildes y con guiones:

```
git switch -c maria-camila-ruiz-gomez
```

3. Abra la carpeta `rickandmorty` en VS Code y ábrala con **Live Server**. Debe ver la lista de personajes. Pulse una tarjeta: llega a una página en blanco. **Ahí empieza el parcial.**

4. **Trabaje en su rama y haga commits mientras avanza**, mínimo tres, con mensajes claros en español.

5. **Suba su rama a GitHub** la primera vez con:

```
git push -u origin maria-camila-ruiz-gomez
```

y después, cada vez que haga commits, con `git push`. **Súbala cada vez que tenga algo funcionando**, no solo al final.

Lo que tiene que hacer la ficha

1. Maquetación (HTML y CSS)

- Un encabezado con el título y un enlace **«← Volver a la lista»**.
- La imagen y los datos del personaje, **uno al lado del otro** en un computador y **uno debajo del otro** en un celular de 375 px, sin desplazamiento horizontal.
- **Unidades relativas** (`rem`, `em`, `%`, `fr`) en el CSS. **No se usan píxeles (`px`).**
- Etiquetas con significado (`header`, `main`, `article`, `section`, `nav`, `ul`…), no solo `div`.

2. Datos (JavaScript)

- **Leer el número del personaje de la dirección** de la página (`character.html?id=7`).
- Pedir **ese** personaje a la API y mostrar: **imagen, nombre, estado, especie, género, origen, ubicación y cuántos episodios tiene.**

- El título de la pestaña del navegador debe ser **el nombre del personaje**.
- Nada de la ficha puede estar escrito a mano en el HTML.

3. Estados

La ficha tiene que decirle algo claro al usuario en cada caso:

Caso	Qué se espera
Cargando	Un mensaje o un spinner mientras llegan los datos. Debe **desaparecer siempre** al terminar, haya salido bien o mal
El personaje no existe (por ejemplo `?id=99999`)	Un mensaje que lo diga con claridad
Falta el número, o no es un número (`character.html` o `?id=abc`)	Un mensaje distinto, **sin pedir nada a la API**
No hubo conexión	Un mensaje distinto de los anteriores

4. Los episodios

Debajo de la ficha, la **lista de todos los episodios** en los que aparece el personaje: **código y nombre** de cada uno (por ejemplo, `S01E01 · Pilot`).

- El personaje solo trae las **direcciones** de sus episodios, no los datos. Rick tiene 51.
- **Todos los episodios deben pedirse con UNA sola petición**, no una por episodio. *La documentación de la API explica cómo.*
- Cuidado: la API contesta con **un objeto** si se pide un episodio, y con **una lista** si se piden varios. Pruebe con un personaje que salga en un solo episodio (el número 6, por ejemplo).
- Si los episodios fallan, **la ficha del personaje no puede desaparecer**.

Reglas

- **El código se escribe en inglés** (nombres de variables y funciones). **Los textos que ve el usuario, en español.**
- **Se puede consultar:** internet, documentación, sus apuntes y IA. **Con una condición:** al entregar, el docente señala **una línea al azar** de su código y usted la explica. **Si no puede explicarla, esa parte no cuenta.**
- Cada uno trabaja en su máquina. No se pide ni se da código a otra persona.
- Abra la página con **Live Server**, no con doble clic.
- Puede crear los archivos que necesite (por ejemplo `js/character.js`).
- Puede consultar la **Chuleta de Git** para los comandos.

Cómo se entrega

1. Haga su último commit en **su rama** y súbalo con `git push`.
2. Compruebe en GitHub, en el selector de ramas, que **su rama aparece con su último commit**.
3. **No se recibe nada por Moodle.** Lo que, cuando el docente cierre el examen, no esté subido a su rama en GitHub no cuenta. Es mejor entregar una ficha incompleta que no entregar.

4. **No haga commits en `main` ni toque la rama de otra persona.**

Cómo se califica

Criterio	Puntos
Maquetación: HTML con significado, ficha que se adapta al celular y CSS sin píxeles	1.0
Datos: número leído de la dirección, personaje pedido a la API, los ocho datos mostrados y título de la pestaña	1.0
Estados: cargando, no existe, número ausente o inválido, y sin conexión, cada uno con su mensaje	1.0
Episodios: lista con código y nombre, en una sola petición, que funcione con uno y con muchos, y que no borre la ficha si falla	1.5
Código y rama: rama con su nombre completo subida a GitHub, commits con su nombre como autor (al menos tres, con mensajes claros), nombres en inglés y sin código sobrante	0.5
	5.0

> **Una advertencia:** las tres primeras partes (maquetación, datos y estados) suman **3.0**. Los episodios, **1.5** más, son la parte difícil. Cuando le funcione lo básico, **pase a los episodios**, y no embellezca: **rómpalo.** Pruebe `?id=99999`, `?id=abc` y apague el internet.